

CADERNO DE PROBLEMAS

16/05/2018 – 14h às 17h30

Leia atentamente estas instruções antes de iniciar a resolução dos problemas. Este caderno é composto de 6 problemas, sendo que 2 deles estão descritos em inglês.

1. Não é permitido o uso de material impresso, livros, apostilas e dicionários. Apenas é permitido o uso de lápis, caneta, lapiseira, borracha, régua e papel para rascunho (o papel é fornecido pela comissão organizadora). O acesso à internet é bloqueado durante a realização da prova. A ajuda do ambiente de desenvolvimento como C, C++ ou Java pode ser utilizada.
2. A correção das resoluções dos problemas será automatizada, por meio do sistema de submissão eletrônica BOCA, tendo como base os resultados obtidos a partir de uma série de execuções dos algoritmos submetidos pelas equipes.
3. Siga atentamente as exigências da tarefa quanto ao formato da entrada e da saída do seu algoritmo. Não implemente nenhum recurso gráfico nas suas soluções (janelas, menus, etc.), nem utilize qualquer rotina para limpar a tela ou posicionar o cursor.
4. Os problemas não estão ordenados por ordem de dificuldade neste caderno. A sugestão é de que procure resolver primeiro os problemas mais fáceis.
5. Utilize os nomes indicados nos problemas para nomear os seus arquivos-fonte, de acordo com a linguagem de programação utilizada.
6. Os problemas devem ser resolvidos utilizando o raciocínio entrada-processamento-saída, ou seja, não é necessário armazenar toda a saída para depois exibi-la.

Observações:

Não utilize arquivos para entrada ou saída. Todos os dados devem ser lidos da entrada padrão (teclado) e escritos na saída padrão (tela). Utilize as funções padrão para entrada e saída de dados:

- em C: `scanf`, `getchar`, `printf`, `putchar`;
- em C++: as mesmas de C ou os objetos `cin` e `cout`;
- em Java:
```
Scanner sc = new Scanner(); int i = sc.nextInt();
String s = sc.next(); float f = sc.nextFloat();
System.out.println(); System.out.printf("%d",i);
```

Em C ou C++: use `sprintf(str,"%d",num);` em vez de `itoa(num,str,10);`

Em Java: não utilize `package`.

Problema

A

Um vencedor (vermelha)

Arquivo fonte: *vencedor.c*, *vencedor.cpp* ou *vencedor.java*

O vencedor do jogo de cartas popular em Fipplândia é determinado de acordo com as seguintes regras. Se no final do jogo houver apenas um jogador com o número máximo de pontos, ele é o vencedor. A situação se torna mais difícil se o número de jogadores for maior que um. Durante cada rodada um jogador ganha ou perde um determinado número de pontos. No decorrer do jogo, o número de pontos é registrado na linha `"nome pontuação"`, onde nome é o nome de um jogador, e pontuação é o número de pontos ganhos nesta rodada, que é um número inteiro. Se a pontuação for negativa, isso significa que o jogador perdeu na rodada. Então, se dois ou mais jogadores tiverem o número máximo de pontos (digamos, é igual a *m*) no final do jogo, quem ganha é apenas aquele que marcou os *m* pontos primeiro. Inicialmente, cada jogador tem 0 pontos. É garantido que no final do jogo pelo menos um jogador tenha um número positivo de pontos.

Entrada

A primeira linha contém um número inteiro n ($1 \leq n \leq 1000$), n é o número de jogadas em uma rodada. Em seguida, tem-se n linhas, contendo as informações sobre a rodada no formato `"nome pontuação"` em ordem cronológica, onde nome é uma cadeia de letras minúsculas com o tamanho de 1 a 32 e pontuação é um número inteiro entre –1000 e 1000, inclusive.

O final da entrada é indicado por um caso em que $n = 0$.

A entrada deve ser lida da entrada padrão.

Saída

Imprima o nome do vencedor.

A saída deve ser escrita na saída padrão.

Exemplo de entrada	**Saída para o exemplo de entrada**

`3`
`cotozica 3`
`pandur 5`
`cotozica 2`
`3`
`pandur 3`
`pandur 2`
`cotozica 5`
`0`
	`pandur`
	`pandur`

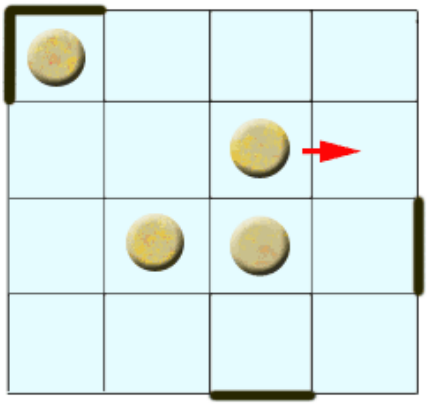
Problema

B

Torta (amarela)

Arquivo fonte: *torta.c*, *torta.cpp* ou *torta.java*

Cotozica e Pandur jogam o seguinte jogo. Existem k tortas nas células de um tabuleiro $n \times m$. A cada rodada, Cotozica move uma torta para a célula vizinha. Se a torta estiver na borda do tabuleiro, Cotozica pode movê-la para fora do tabuleiro, pegar a torta e ganhar. Após o movimento de Cotozica, Pandur retira uma borda do tabuleiro de comprimento 1 (entre duas célula do tabuleiro) para que Cotozica não seja capaz de mover a torta para fora do tabuleiro por essa borda. A questão é: Cotozica vai ganhar este jogo? Supomos que ambos os jogadores sigam a estratégia ideal.



Entrada

A primeira linha contém 3 inteiros, separados por espaço: $1 \leq n, m \leq 100$ (dimensões do tabuleiro) e $0 \leq k \leq 100$ (número de tortas). Cada uma das próximas k linhas contém 2 inteiros, separados por espaço: $1 \leq x \leq n$, $1 \leq y \leq m$ (coordenadas da torta correspondente). Pode haver mais de uma torta em uma célula.

O final da entrada é indicado por um caso em que $n = 0$, $m = 0$ e $k = 0$.

A entrada deve ser lida da entrada padrão.

Saída

Imprima na Saída apenas uma palavra `SIM`, se Cotozica ganha, `NAO`, caso contrário.

A saída deve ser escrita na saída padrão.

Exemplo de entrada	**Saída para o exemplo de entrada**

`2 2 1`
`1 2`
`3 4 0`
`100 50 2`
`50 25`
`50 25`
`0 0 0`
	`SIM`
	`NAO`
	`NAO`

Death Stars (azul)

Problema C

Source file: *stars.c*, *stars.cpp* or *stars.java*

The stardate is 1977 and the science and art of detecting Death Stars is in its infancy. Princess Dayn has received information about the stars in the nearby solar system from the Rebel spies and now, to help her identify the exact location of the Death Star, she needs to know whether this information is correct.

Two rebel spies have provided her with the maps of the solar system. Each map is an $N \times N$ grid, where each cell is either occupied by a star or empty. To see whether the information is correct, Dayn needs to know whether the two maps are of the same solar system, or if possibly one of the spies is actually an Empire double agent, feeding her false information.

Unfortunately, spies may have accidentally rotated a map by 90, 180, or 270 degrees, or flipped it along the vertical or the horizontal axis, before delivering it to Dayn. If Dayn can rotate or flip the maps so that two of them become identical, then those maps are of the same solar system. Otherwise, there are traitors in the Rebel ranks! Help Dayn find out.

Input

The first line of the input contains one number N ($1 \leq N \leq 10$) that is the dimension of each map. Next N lines each contain N characters, depicting the first map: `'X'` indicates a star, while `'O'` indicates an empty quadrant of space. Next N lines each contain N characters, depicting the second map in the same format.

The end of input is indicated by a case in which N = 0.

The input must be read from standard input.

Output

The line of output should contain the word `Yes` if the maps are identical, or `No` if it is impossible to match them by performing rotations and translations.

Please see the sample input and sample output for exact format.

The output must be written to standard output.

Sample input	Output for the sample input
4 XOOO XXOO OOOO XXXX XOOO XOOO XOXO XOXX 2 XX OO XO OX 0	Yes No

Equation (verde)

Problema D

Source file: *equation.c*, *equation.cpp* or *equation.java*

You are given an equation:

$$Ax^2 + Bx + C = 0$$

Your task is to find the number of distinct roots of the equation and print all of them in ascending order.

Input

The first line contains three integer numbers A, B and C ($-10^5 \le A, B, C \le 10^5$). Any coefficient may be equal to 0.

The end of input is indicated by a case in which A = 0, B = 0 and C = 0.

The input must be read from standard input.

Output

In case of no roots print the only integer 0. In other cases print the number of root on the first line and the roots on the following lines in the ascending order. Print roots with 5 digits after the decimal point.

The output must be written to standard output.

Sample input	**Output for the sample input**
1 -5 6 0 3431 43123 0 0 0	2 2.00000 3.00000 1 -12.56864

Medalhão (rosa)

Problema E

Arquivo fonte: *medalhao.c*, *medalhao.cpp* ou *medalhao.java*

Harry soube por Dumbledore que o medalhão de Salazar Slytherin é uma horcrux. Este medalhão estava presente em 12 Grimmauld Place, a casa da mãe de Sirius Black. Foi roubado de lá e agora está presente no Ministério da Magia no escritório de Dolorous Umbridge, ex-defesa Contra as Artes das Trevas de Harry.

Harry, Ron e Hermione estão se infiltrando no Ministério. Ao chegar ao escritório de Umbridge, eles observaram um bloqueio de código com um quebra-cabeça, pedindo-lhes para calcular a contagem de números mágicos entre dois inteiros l e r (ambos inclusive).

Harry lembrou-se de seu tempo de detenção com Umbridge onde definiu um número mágico como um número que, quando convertido para uma dada base b, todos os dígitos de 0 a $b - 1$ aparecem o mesmo número de vezes em sua representação sem zeros à esquerda.

Você precisa responder q consultas para desbloquear o escritório. Cada consulta tem três inteiros b_i, l_i e r_i, a base e o intervalo para o qual você tem que encontrar a contagem de números mágicos.

Entrada

A primeira linha de entrada contém q ($1 \leq q \leq 10^5$) número de consultas. Cada uma das próximas q linhas contêm três inteiros separados por espaço b_i, l_i, r_i ($2 \leq b_i \leq 10$, $1 \leq l_i \leq r_i \leq 10^5$).

O final da entrada é indicado por um caso em que $q = 0$.

A entrada deve ser lida da entrada padrão.

Saída

Você tem que gerar q linhas, cada uma contendo um único inteiro, a resposta para a consulta correspondente.

A saída deve ser escrita na saída padrão.

Exemplo de entrada	**Saída para o exemplo de entrada**
2 2 4 9 3 1 10 2 2 1 100 5 1 100 0	1 2 21 4

Teclado Antigo (branca)

Problema F

Arquivo fonte: *teclado.c*, *teclado.cpp* ou *teclado.java*

Os cientistas descobriram um dispositivo antigo que funciona de uma maneira estranha. O dispositivo tem um teclado e uma fita de saída. O teclado tem 26 teclas, com os símbolos `'A'` a `'Z'`. Cada tecla tem um LED (como a tecla *Caps Lock* em alguns teclados). Cada vez que você pressiona uma tecla, o LED acende (muda seu estado de desligado para ligado ou vice-versa). Todos os LEDs estão desligados inicialmente.

Para estudar a saída gravada na fita, consideramos o dispositivo em etapas de tempo discretas. Suponha que estamos no tempo *t*. Se nenhum LED estiver aceso, nenhuma saída será gravada na fita. Se houver *i* LEDs acesos, a *i*-ésima letra do alfabeto é escrita na fita. Por exemplo, se três LEDs estiverem ativados em uma etapa de tempo, uma letra `'C'` será gravada na fita. Este processo se repete a cada passo do tempo.

Você foi solicitado a escrever um programa que simula o antigo dispositivo.

Entrada

A entrada contém vários casos de teste. A primeira linha da entrada contém *t*, o número de casos de teste que se seguem. Cada um dos blocos *t* a seguir descreve um caso de teste.

A primeira linha de cada bloco contém um inteiro n ($0 \leq n \leq 26$). Depois disso, existem n linhas, cada uma contendo uma letra maiúscula do alfabeto, seguida por dois inteiros a e b, ($0 \leq a < b \leq 1000$). A letra maiúscula mostra a tecla pressionada. O número a é o primeiro passo de tempo no qual a tecla é pressionada e o número b é o segundo passo de tempo no qual a tecla é pressionada. Durante o intervalo $a, a + 1, ..., b - 1$, o LED da tecla está aceso. Você pode assumir que, em cada caso de teste, essas letras são distintas.

A entrada deve ser lida da entrada padrão.

Saída

Para cada caso de teste, imprima uma linha contendo a string de saída que está escrita na fita.

A saída deve ser escrita na saída padrão.

Exemplo de entrada	**Saída para o exemplo de entrada**
2 2 X 2 6 Y 4 9 3 A 1 5 B 4 8 C 9 10	AABBAAA AAABAAAA